Object-Oriented Modelling

UNIT – 1

Chapter 1

Class Modelling

Modeling as a Design Technique

Modeling

- Modeling in **Object-Oriented Modeling (OOM)** refers to the process of creating abstract representations (models) of real-world entities, systems, or problems using object-oriented principles.

- These models help in understanding, designing, and developing software systems by visualizing how different components interact.

Key Aspects of Modeling in OOM

1. **Abstraction:** Simplifies complex systems by focusing on essential details while ignoring irrelevant ones.
2. **Encapsulation:** Groups data and behaviour (methods) within objects to promote modularity.
3. **Inheritance:** Allows new objects (subclasses) to inherit properties and behaviours from existing objects (superclasses).
4. **Polymorphism:** Enables objects to be treated as instances of their parent class, improving flexibility and code reusability.

Purpose of Modeling

1. Testing a physical entity before building it.

- Testing a physical entity before building it is known as **modeling and simulation** or **prototyping**. In the context of **Object-Oriented Modeling (OOM)**, this concept is often applied through **digital models** that represent real-world physical entities before actual construction or manufacturing.

- **Benefits of Testing Before Building**

a) **Cost Reduction:** Avoids expensive mistakes in real-world implementation.

b) **Safety:** Helps in identifying potential failures before actual deployment.

c) **Optimization:** Allows fine-tuning of designs for better efficiency.

d) **Faster Development:** Reduces trial-and-error in physical prototyping.

2. Communication with customers

- Effective communication with customers is a crucial part of **modeling a system** in software development. It ensures that the final product meets the customer's needs and expectations.
- In **Object-Oriented Modeling (OOM)**, various techniques and models are used to facilitate this communication such as usecase diagrams, activity diagrams, etc.
- **Benefits of Effective Communication in System Modeling**

a) **Reduces misunderstandings** between developers and customers.

b) **Ensures the system meets business needs** before development starts.

c) **Improves customer satisfaction** with regular feedback loops.

d) **Saves time and cost** by identifying issues early.

3. Visualization.

- Visualization in **Object-Oriented Modeling (OOM)** refers to the graphical representation of objects, their relationships, behaviors, and interactions within a system.
- It helps developers, designers, and stakeholders understand and communicate system structures and workflows efficiently.
- **Benefits of Visualization in OOM**

a) **Enhances communication** between technical and non-technical stakeholders.

b) **Reduces ambiguity** by providing a **clear graphical representation**.

c) **Improves system design** by identifying **flaws early**.

d) **Speeds up development** by serving as a **blueprint** for coding.

4. Reduction of complexity.

- In **Object-Oriented Modeling (OOM)**, reducing complexity is essential to make the system more understandable, maintainable, and scalable. OOM provides several techniques to simplify complex systems while retaining essential functionality.

- **Benefits of Complexity Reduction**

Improves Code Readability – Easier for developers to understand and modify.

Enhances Maintainability – Changes in one module don't break the entire system.

Increases Reusability – Promotes reuse of existing components.

Speeds Up Development – Simplifies testing and debugging.

Abstraction

- Abstraction in **Object-Oriented Modeling (OOM)** is the process of **hiding unnecessary details** and exposing only essential features of an object. It helps in managing complexity by focusing on what an object **does** rather than **how** it does it.

- **Key Principles of Abstraction**

1. Focus on Relevant Information

Only the essential attributes and behaviors of an object are exposed.

Example: A Car object has a startEngine() method, but the user doesn't need to know the internal working of the engine.

2. Hiding Internal Implementation

Implementation details are encapsulated and hidden from external access.

Example: A BankAccount class provides deposit() and withdraw() methods but hides how the balance is updated in the database.

3. Providing a Simple Interface

Users interact with an abstract interface without dealing with complexity.

Example: A sendMessage() function in a Messaging App handles all underlying networking without exposing it to the user.

The Three Models

A system is best modeled from three related viewpoints, each capturing essential aspects but all necessary for a complete description.

- The **class model** represents the static, structural, and data aspects.
- The **state model** captures temporal, behavioral, and control aspects.
- The **interaction model** focuses on object collaboration and interaction.

1. Class Model

The **Class Model** represents the **static structure** of a system. It defines **classes, attributes, operations (methods), and relationships** between objects. This model captures the **data** aspects of a system and forms the foundation for object-oriented design.

Key Components of the Class Model

Classes: Represent real-world entities.

Example: Book, Member, Librarian in a library system.

Attributes: Define properties of a class.

Example: Book has title, author, ISBN.

Methods (Operations): Define behaviour of a class.

Example: Book class has borrow(), returnBook().

Relationships:

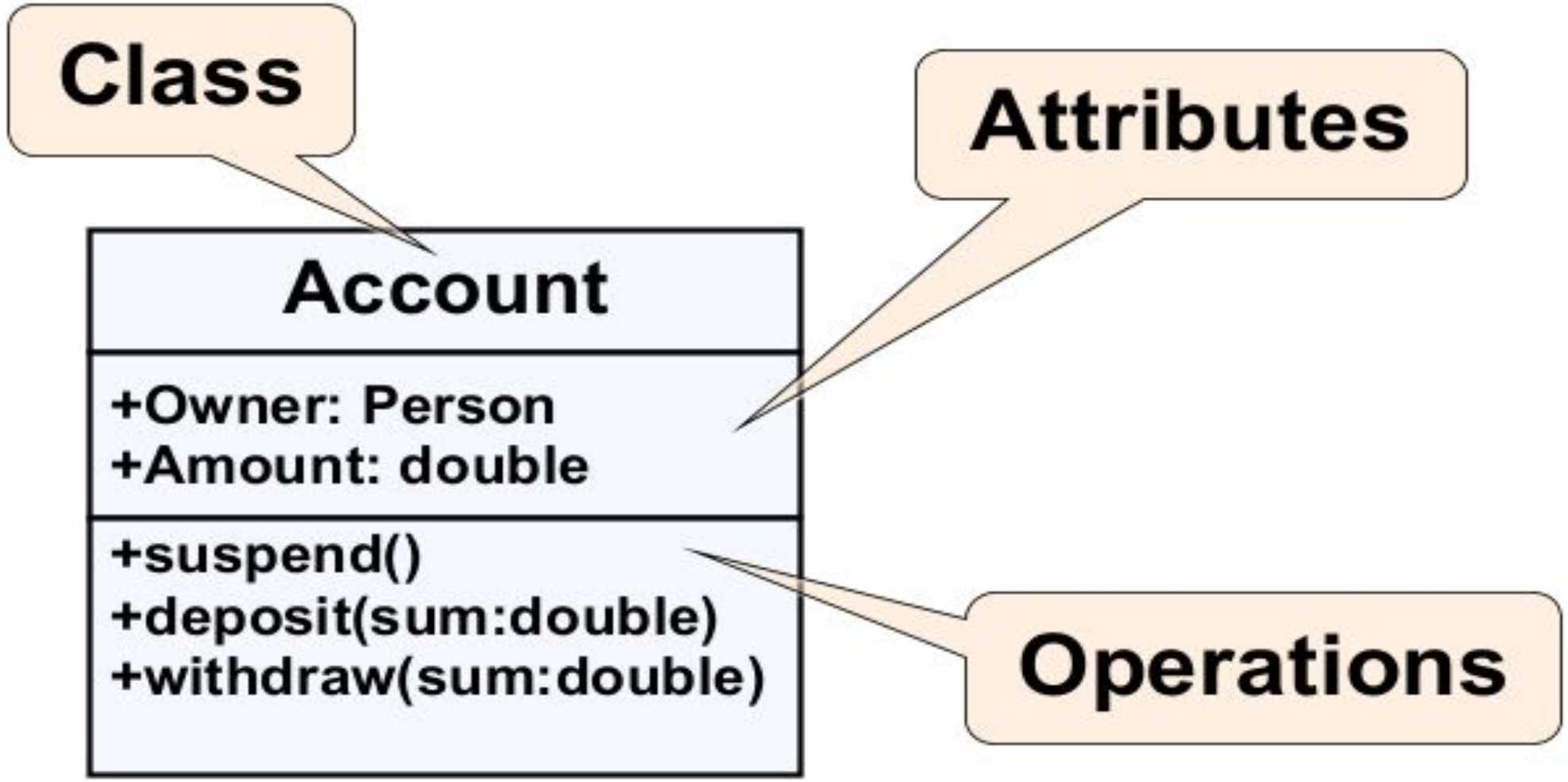
Association: A connection between two classes (e.g., A Member borrows a Book).

Aggregation: A whole-part relationship (e.g., A Library contains multiple Books).

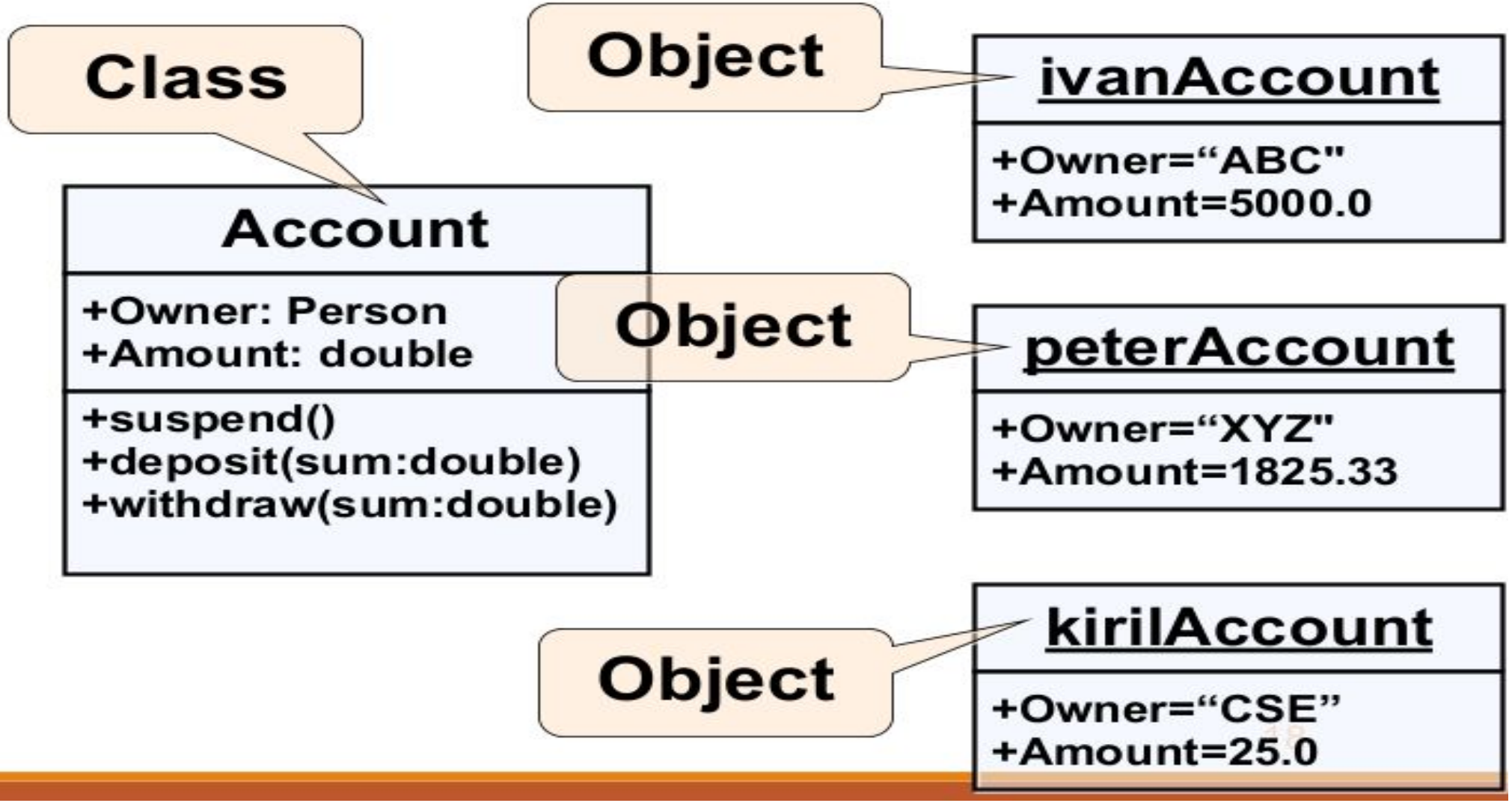
Composition: A strong whole-part relationship (e.g., A Car has an Engine that cannot exist independently).

Inheritance: A subclass inherits properties from a superclass (e.g., Car and Bike inherit from Vehicle).

Classes - Example



Classes and Objects – Example



Class Modelling

a. Object and Class Concepts

- In Object-Oriented Modeling (OOM), an object is a fundamental concept that represents an instance of a class.

- It is a self-contained unit that consists of:

Attributes (or Properties): These are the data or state variables that define the characteristics of the object. For example, if the object is a "Car," attributes might include its color, make, model, and speed.

Methods (or Behaviors): These are the functions or operations that an object can perform. Continuing with the "Car" example, methods might include actions like start(), accelerate(), or brake().

b. Classes

- An object is an instance—or occurrence—of a class. A class describes a group of objects with the same properties (attributes), behavior (operations), kinds of relationships, and semantics.

- Classes often appear as common nouns and noun phrases in problem descriptions and discussions with users.

- In summary, a **class** is like a mold, and the **objects** are the individual instances created from that mold, each containing their own data but following the same structure and behavior defined by the class.

c. Class Diagrams

- Class diagrams provide a graphic notation for modelling classes and their relationships, thereby describing possible objects

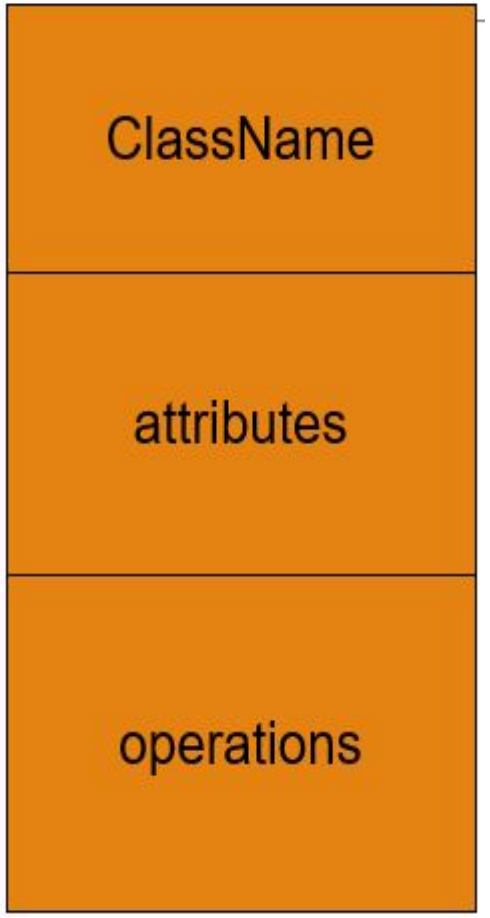
- It shows the classes, their attributes, methods, and the relationships between classes in an object-oriented system.

- Class diagrams are essential in Object-Oriented Modeling (OOM) as they provide a blueprint of the system’s design.

Key Components of a Class Diagram:

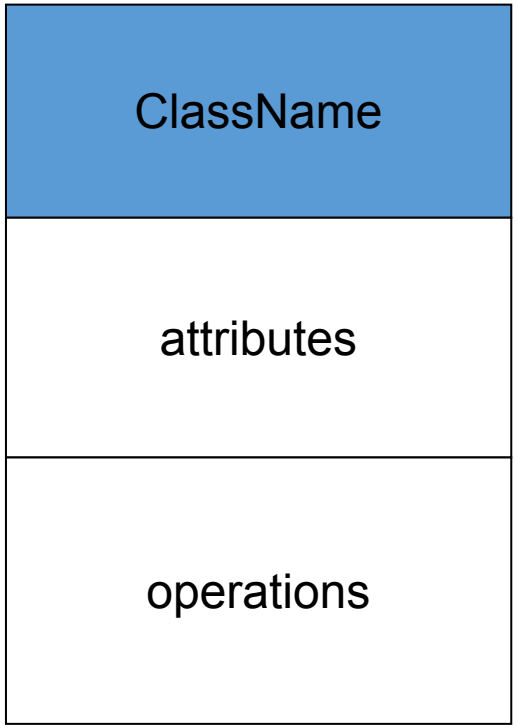
i. Class

- Class is a description of a set of objects that share the same attributes, operations, relationships, and semantics.
- Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations in separate, designated compartments.



ii. Class Names

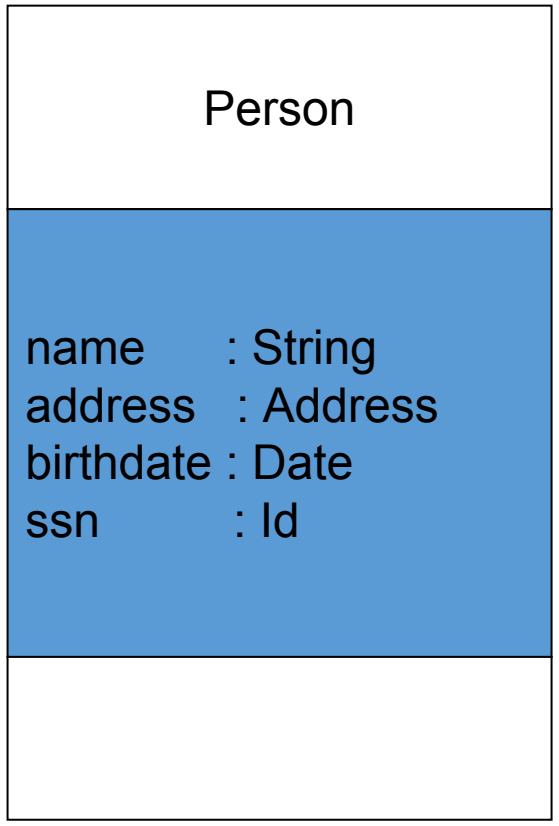
- The **name** of the class is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.



iii. Class Attributes

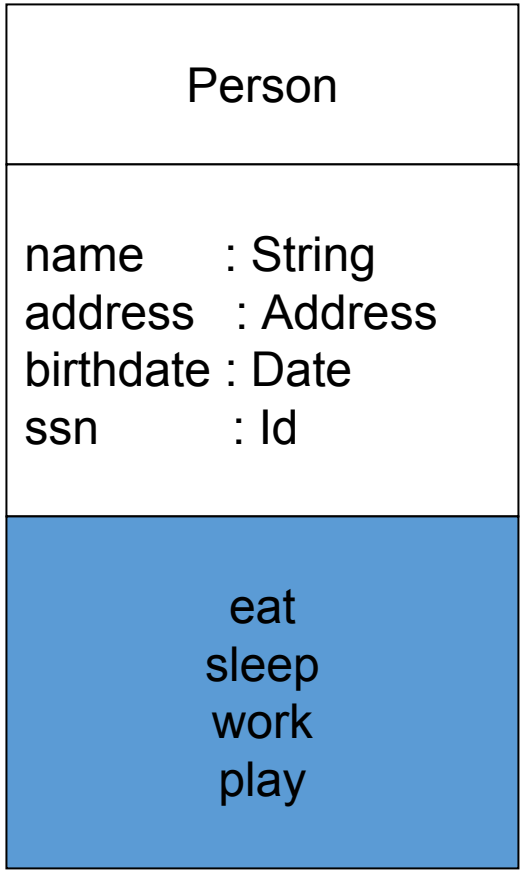
- An ***attribute*** is a named property of a class that describes the object being modeled.

- In the class diagram, attributes appear in the second compartment just below the name-compartment.

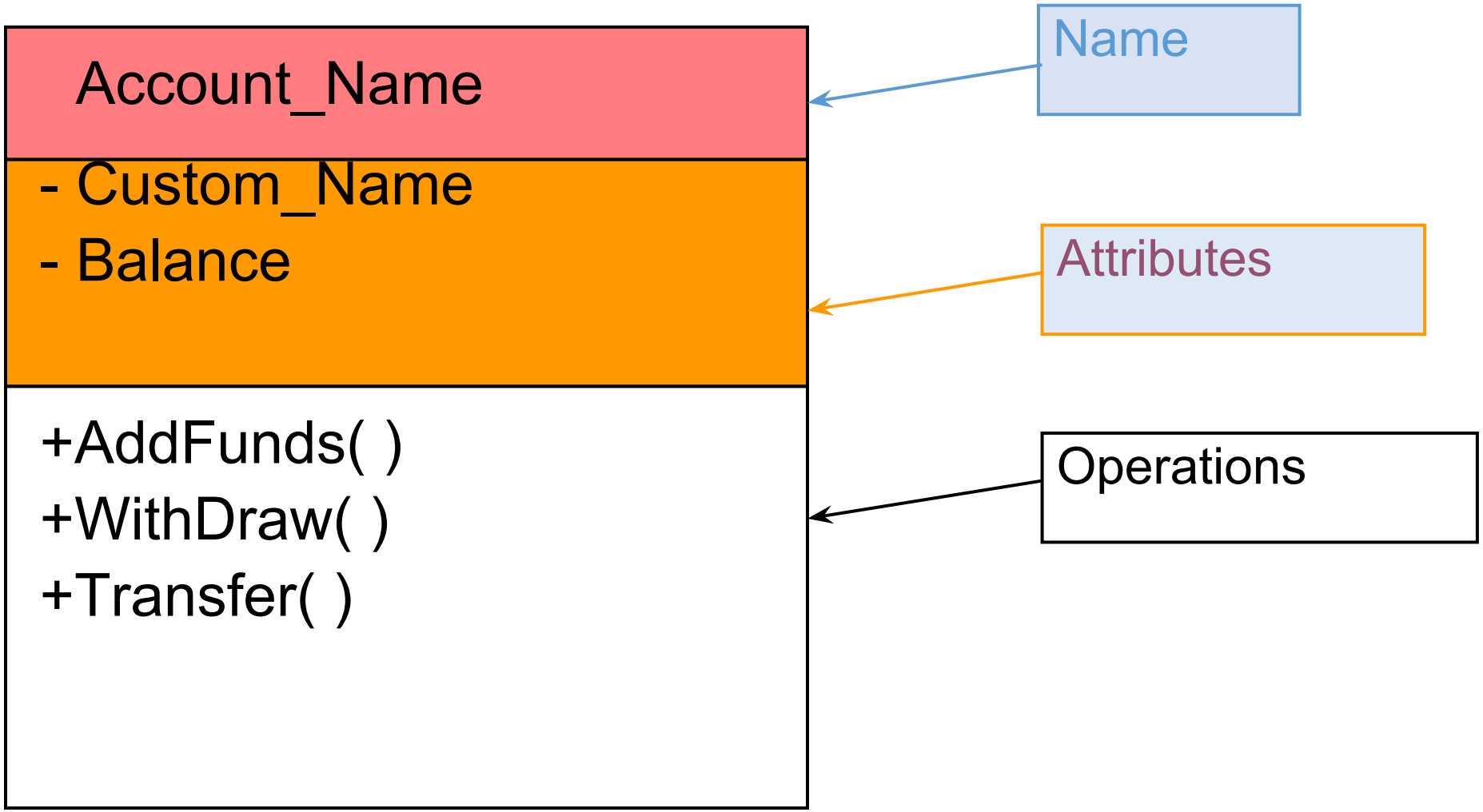


iv. Class Operations

- ***Operations*** describe the class behavior and appear in the third compartment.



An example of Class



d. Values and Attributes

- A value is a piece of data. You can find values by examining problem documentation for examples.

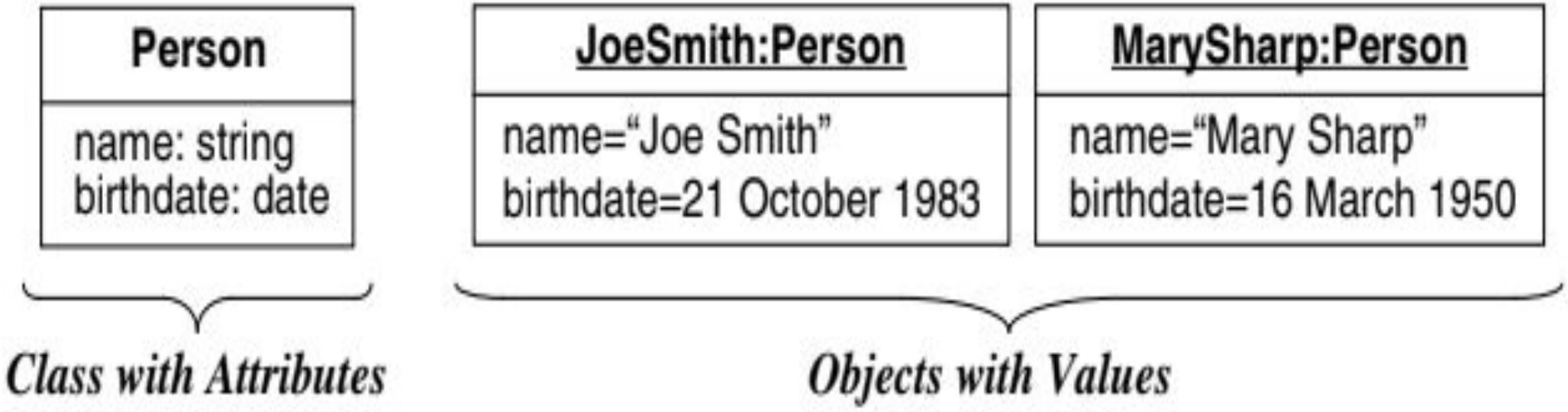
- An attribute is a named property of a class that describes a value held by each object of the class.

- Name, birthdate, and weight are attributes of Person objects. Color, model Year, and weight are attributes of Car objects. Each attribute has a value for each object. For example, attribute birthdate has value “21 October 1983” for object JoeSmith.

- Different objects may have the same or different values for a given attribute. Each attribute name is unique within a class.

- Figure below shows modeling notation. Class Person has attributes name and birthdate.

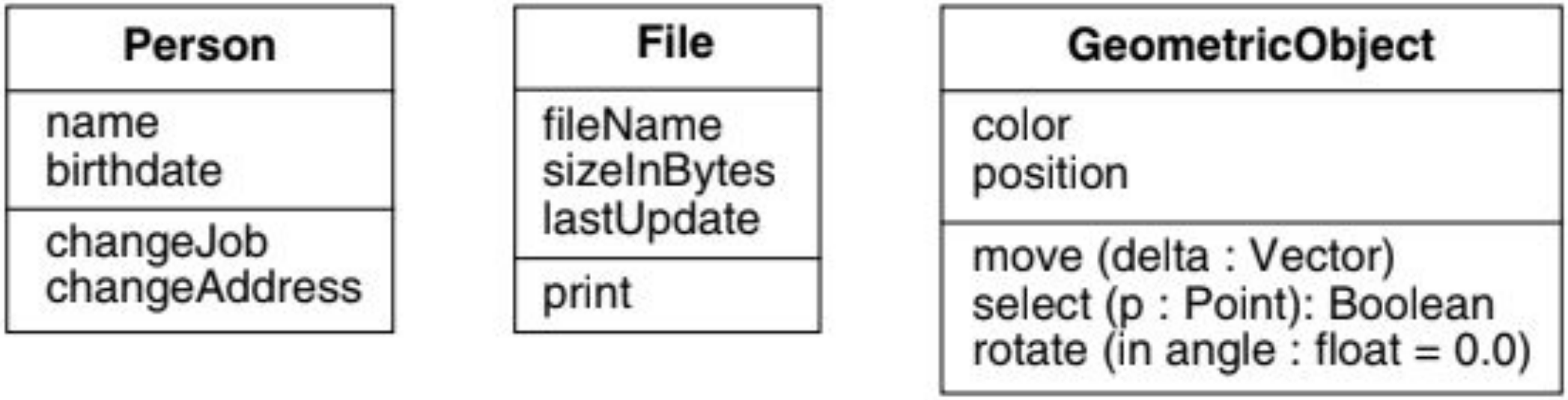
- Name is a string and birthdate is a date. One object in class Person has the value “Joe Smith” for name and the value “21 October 1983” for birthdate. Another object has the value “Mary Sharp” for name and the value “16 March 1950” for birthdate.



e. Operations and Methods

- An operation is a function or procedure that may be applied to or by objects in a class. Hire, fire, and payDividend are operations on class Company.

- All objects in a class share the same operations.



Link and Association Concepts

- In Object-Oriented Modeling and Design (OOMD), **links** and **associations** are both essential concepts used to represent the relationships between objects or classes in a system.

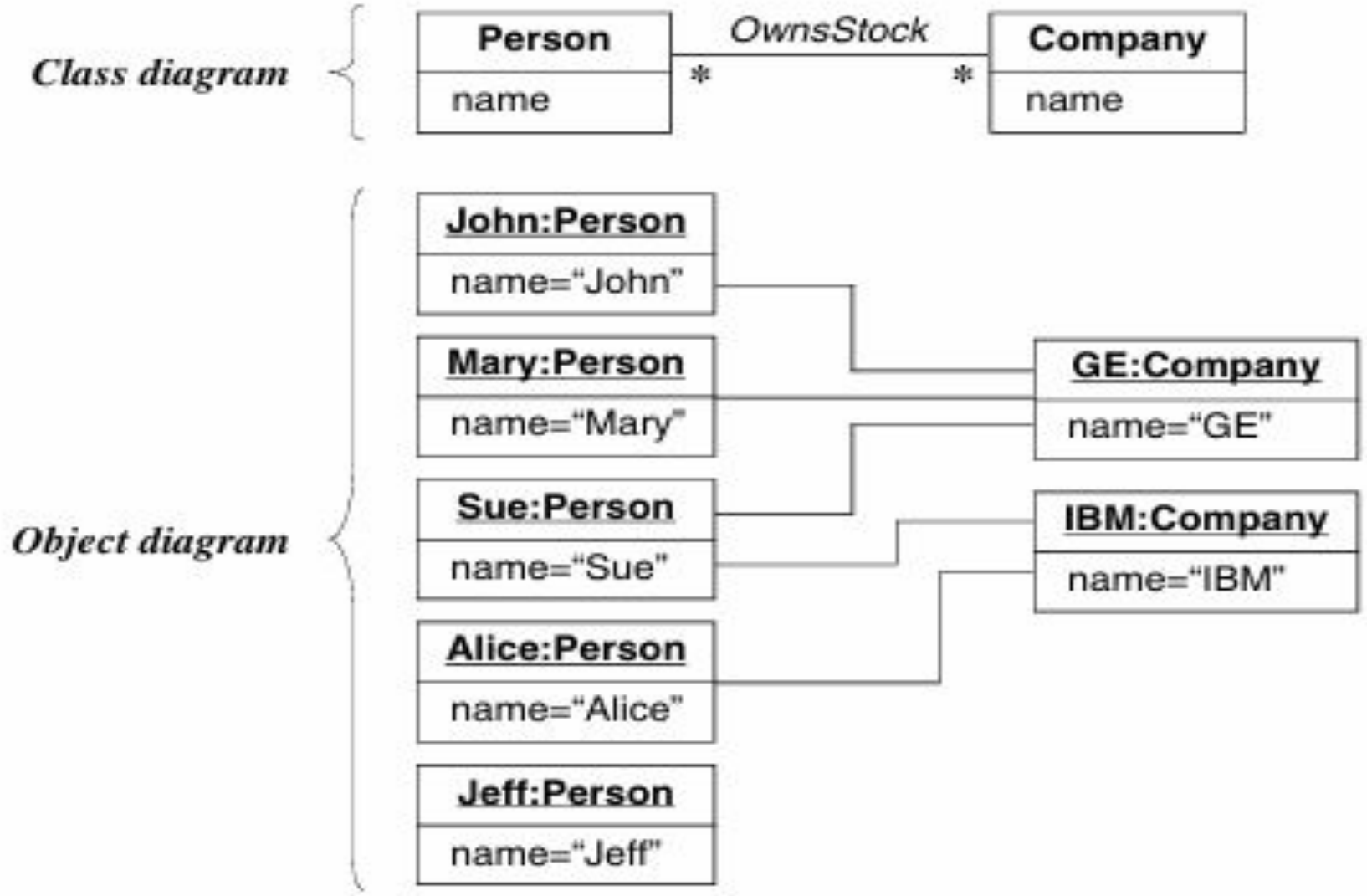
- They help structure how objects collaborate with one another and allow you to design complex systems with clear relationships.

- A **link** is a physical or conceptual connection among objects. For example, Joe Smith Works For Simplex company.

- Most links relate two objects, but some links relate three or more objects.

- An association is a description of a group of links with common structure and common semantics. For example, a person WorksFor a company.

Many-to-many association: An association describes a set of potential links in the same way that a class describes a set of potential objects.



- In the class diagram, a person may own stock in zero or more companies; a company may have multiple persons owning its stock.

- The object diagram shows some examples.
  - ✔ John, Mary, and Sue own stock in the GE company.
  - ✔ Sue and Alice own stock in the IBM company.
  - ✔ Jeff does not own stock in any company and thus has no link.

- The asterisk is a multiplicity symbol. Multiplicity specifies the number of instances of one class that may relate to a single instance of another class and is discussed in the next section.

- The association name is optional, if the model is unambiguous.

- Ambiguity arises when a model has multiple associations among the same classes person works for company and person owns stock in company).

- When there are multiple associations, you must use association names or association end names to resolve the ambiguity.

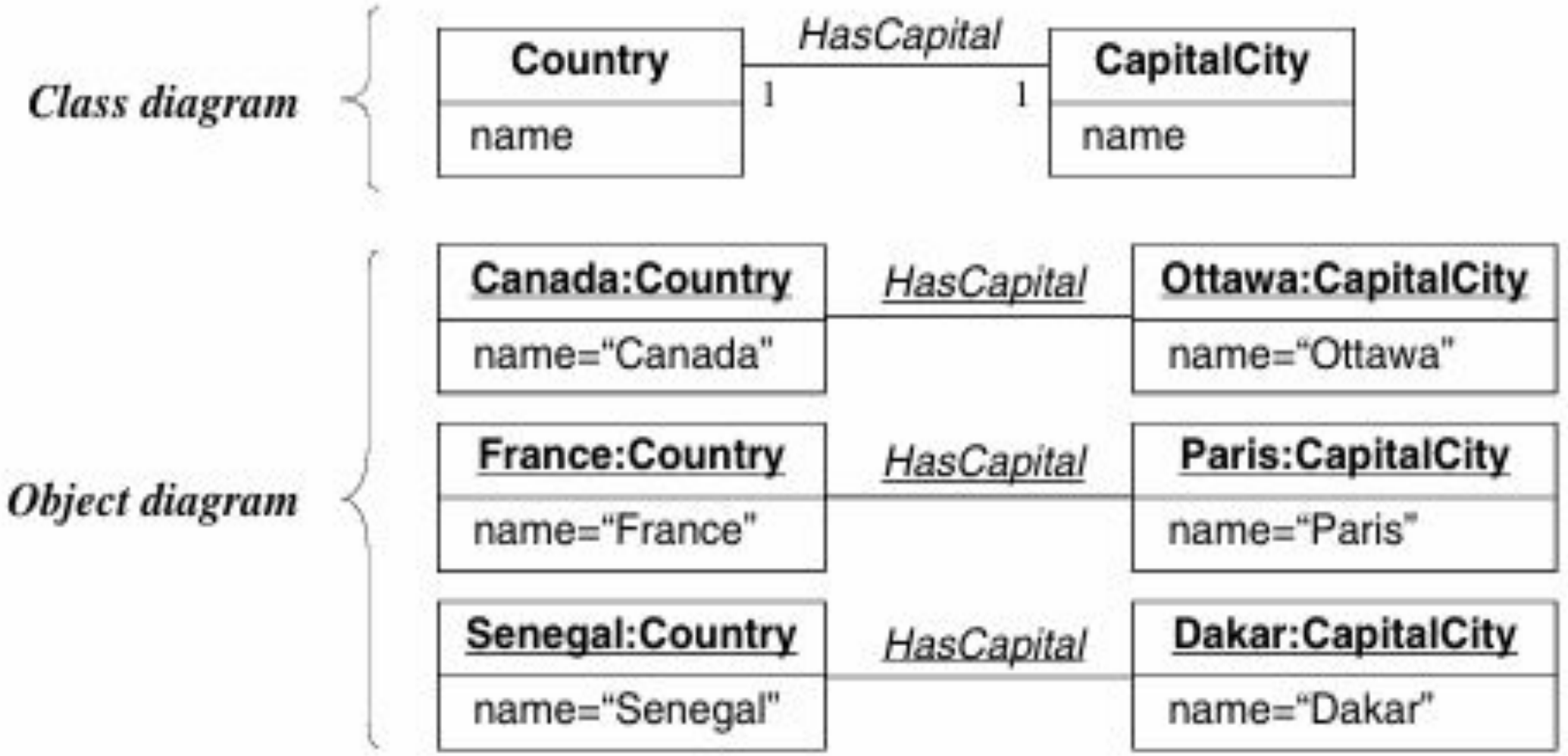
- Associations are inherently bidirectional.

Multiplicity

- Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.
- Multiplicity constrains the number of related objects.
- The literature often describes multiplicity as being “one” or “many,” but more generally it is a (possibly infinite) subset of the nonnegative integers.
- UML diagrams explicitly list multiplicity at the ends of association lines.
- The UML specifies multiplicity with an interval, such as “1” (exactly one), “1..*” (one or more), or “3..5” (three to five, inclusive).
- The special symbol “*” is a shorthand notation that denotes “many” (zero or more).

- The previous figure illustrates many-to-many multiplicity.
  - ✔ A person may own stock in many companies.
  - ✔ A company may have multiple persons holding its stock.
  - ✔ In this particular case, John and Mary own stock in the GE company; Alice owns stock in the IBM company; Sue owns stock in both companies; Jeff does not own any stock.
  - ✔ GE stock is owned by three persons; IBM stock is owned by two persons.

- The next figure shows a one-to-one association and some corresponding links. Each country has one capital city. A capital city administers one country.



One-to-one association: Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.

Common Multiplicities:

- 1..1: One object of the first class is associated with exactly one object of the second class.
- 0..1: One object of the first class can be associated with zero or one object of the second class (optional relationship).
- 1..*: One object of the first class is associated with one or more objects of the second class (mandatory, but can have many).
- 0..*: One object of the first class can be associated with zero or more objects of the second class (optional, but can have many).
- n..m: One object of the first class is associated with a specific number of objects of the second class, between n and m.

Examples of Multiplicity

- **One-to-One Relationship (1..1)**

A Person has one Passport, and a Passport is assigned to exactly one Person.

- **One-to-Many Relationship (1..*)**

A Teacher can teach multiple Students, but each Student is taught by one Teacher.

- **Zero or One (0..1)**

A Person may or may not have an Email, but if they do, they have exactly one.

- **Many-to-Many Relationship (0..*)**

A Student can enroll in multiple Courses, and a Course can have multiple Students.

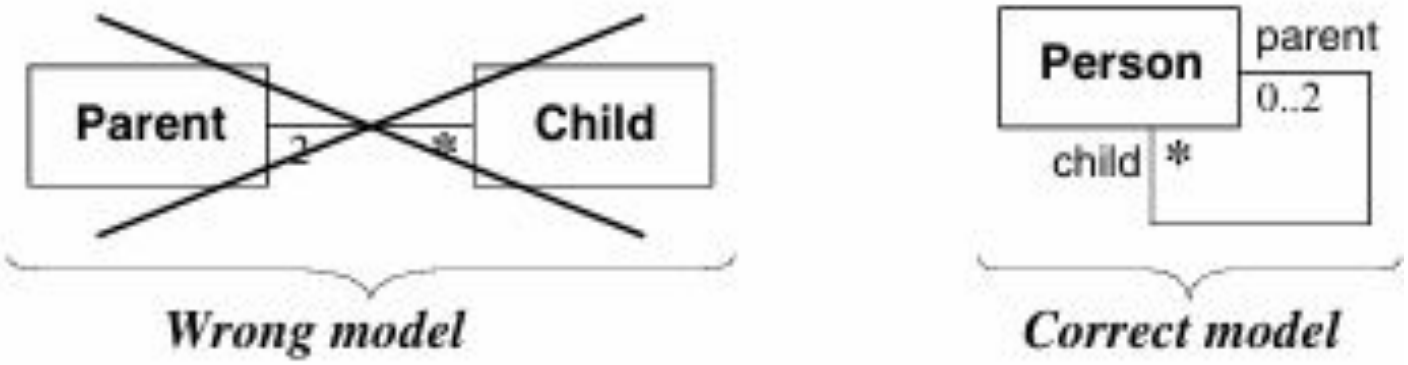
Association End Names

- Association end names often appear as nouns in problem descriptions.
- As Figure below shows, a name appears next to the association end.
- In the figure Person and Company participate in association WorksFor.
- A person is an employee with respect to a company; a company is an employer with respect to a person.
- Use of association end names is optional, but it is often easier and less confusing to assign association end names instead of, or in additi



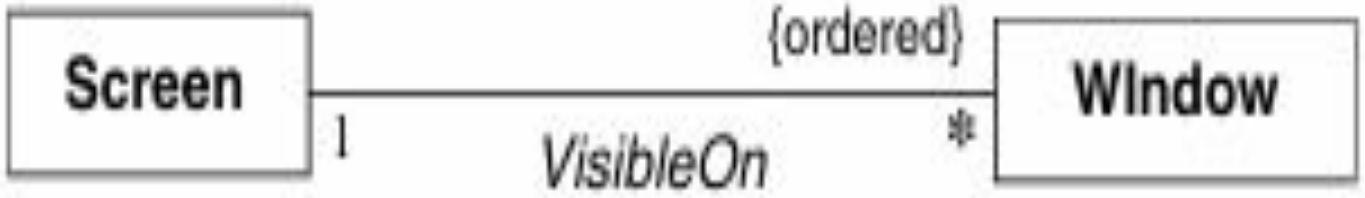
employee	employer
Joe Doe	Simplex
Mary Brown	Simplex
Jean Smith	United Widgets

- Association end names let you unify multiple references to the same class.
- When constructing class diagrams you should properly use association end names and not introduce separate class for each reference, as Figure below shows.
- In the wrong model, two instances represent a person with a child, one for the child and one for the parent.
- In the correct model, one person instance participates in two or more links, twice as a parent and zero or more times as a child.



Ordering

- Often the objects on a “many” association end have no explicit order, and you can regard them as a set.
- Sometimes, however, the objects have an explicit order.
- For example, Figure below shows a workstation screen containing a number of overlapping windows.
- Each window on a screen occurs at most once.
- The windows have an explicit order, so only the top most window is visible at any point on the screen.
- The ordering is an inherent part of the association. You can indicate an ordered set of objects by writing “{ordered}” next to the appropriate association end.

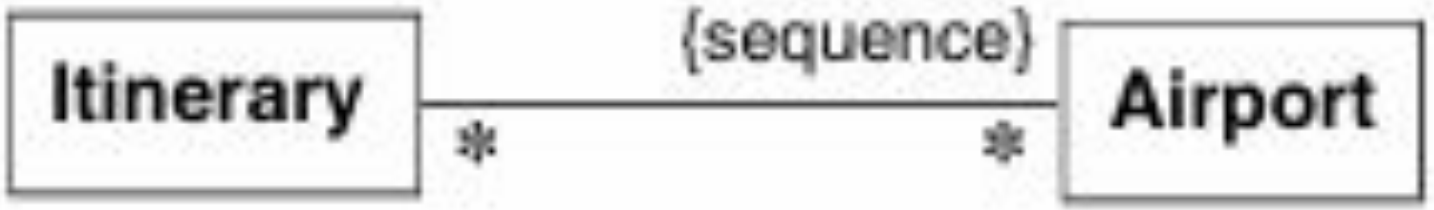


Ordering the objects for an association end. Ordering sometimes occurs for "many" multiplicity.

Bags and Sequences

- Ordinarily a binary association has at most one link for a pair of objects.
- However, you can permit multiple links for a pair of objects by annotating an association end with {bag} or {sequence}.
- A **bag** is a collection of elements with duplicates allowed.
- A **sequence** is an ordered collection of elements with duplicates allowed.

- In Figure below, an itinerary is a sequence of airports and the same airport can be visited more than once.
- Like the {ordered} indication, {bag} and {sequence} are permitted only for binary associations.



- Note that the {ordered} and the {sequence} annotations are the same, except that the first disallows duplicates and the other allows them.

Association Classes

- An **association class** is a special type of class used to represent the relationship between two or more classes.
- It acts as an intermediary between classes that are involved in an **association**.
- Typically, an association class is used when the relationship between objects has attributes or behaviors that are not naturally represented by either of the two associated classes alone.
- In other words, the association class allows you to model additional information or behavior specific to the relationship itself.

Key Characteristics of Association Classes:

- **Represents a relationship**: An association class models a relationship between two or more classes.

- **Has attributes or methods**: Unlike a simple association (which just links two classes), an association class can have its own attributes and methods that describe or manage the relationship between the classes.

- **Can have multiplicity**: An association class can specify how many instances of the relationship can exist between the associated classes.

When to Use an Association Class:

- When you want to **add attributes** to an association.
- When the relationship between classes itself has **behavior** (methods) that should be modeled separately from the associated classes.

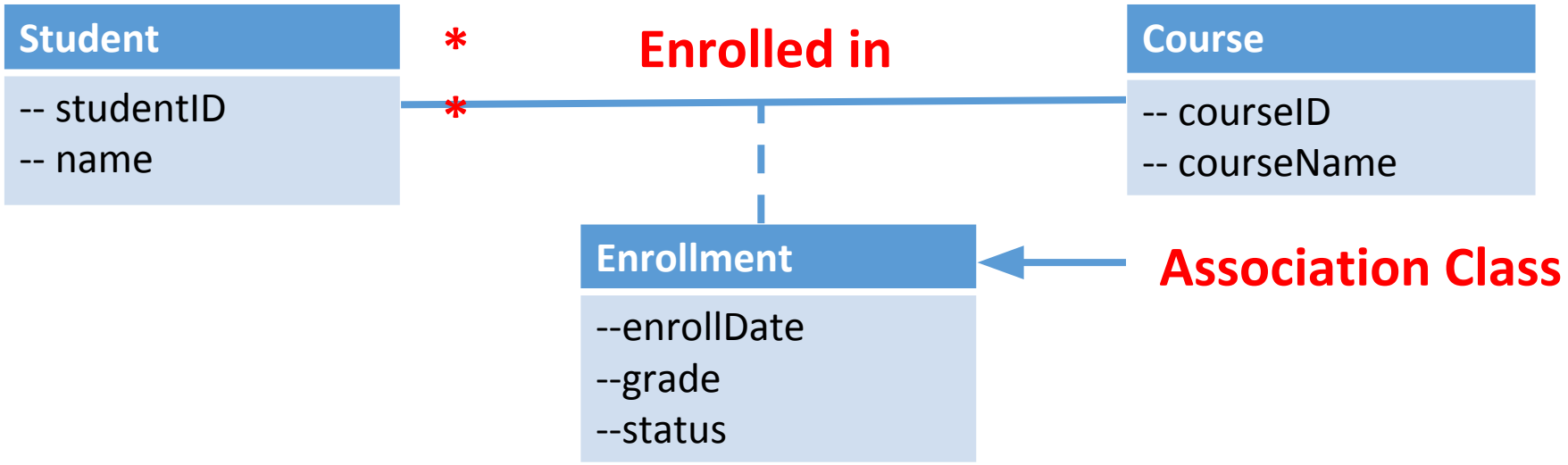
UML Representation of an Association Class:

- The association is usually shown with a solid line connecting the two associated classes.
- The association class is shown as a rectangle attached to this line by a dashed line.
- The association class typically represents the relationship's attributes and methods.

Example:

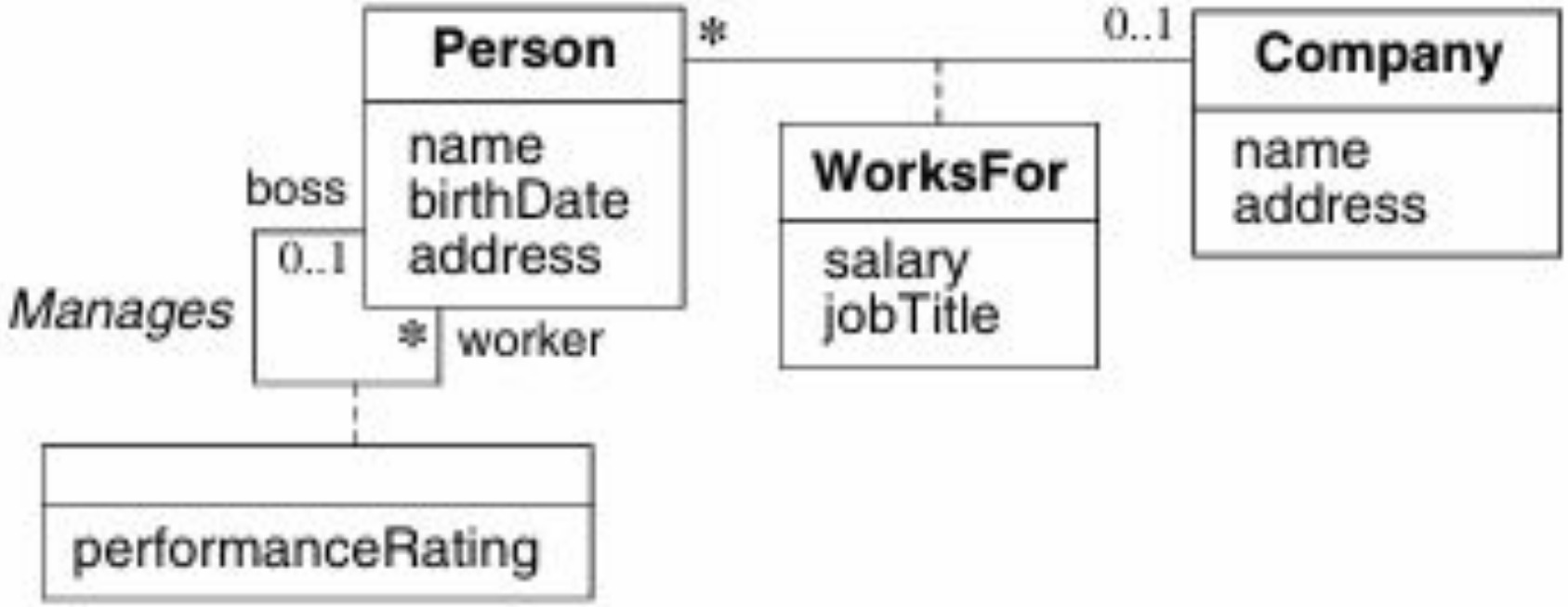
Let's say you are modeling a system where **students** and **courses** are related. A student can enroll in multiple courses, and a course can have multiple students. However, the relationship between students and courses has additional details, such as the enrollment date, grade, or status.

- **Student** and **Course** are the two main classes.
- The **Enrollment** class (the association class) represents the relationship between them.



- Figure below presents attributes for two one-to-many associations. Each person working for a company receives a salary and has a job title.

- The boss evaluates the performance of each worker. Attributes may also occur for one-to-one associations.



- Figure below shows how it is possible to fold attributes for one-to-one and one-to-man associations into the class opposite a “one” end. This is not possible for many-to-many associations. As a rule, you should not fold such attributes into a class because the multiplicity of the association might change.

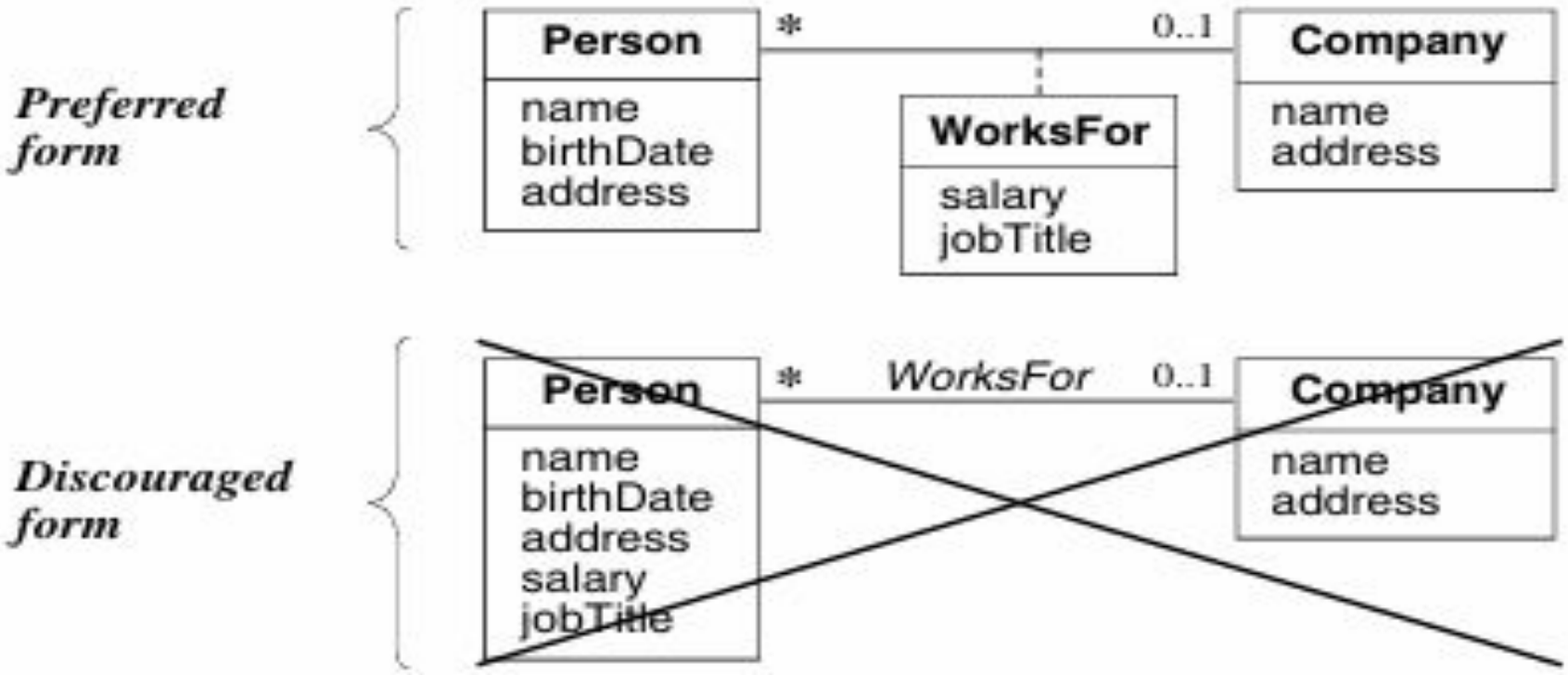


Figure: Proper use of association classes. Do not fold attributes of an association into a class.

- Figure below shows an association class participating in an association. Users may be authorized on many workstations. Each authorization carries a priority and access privileges. A user has a home directory for each authorized workstation, but several workstations and users can share the same home directory.

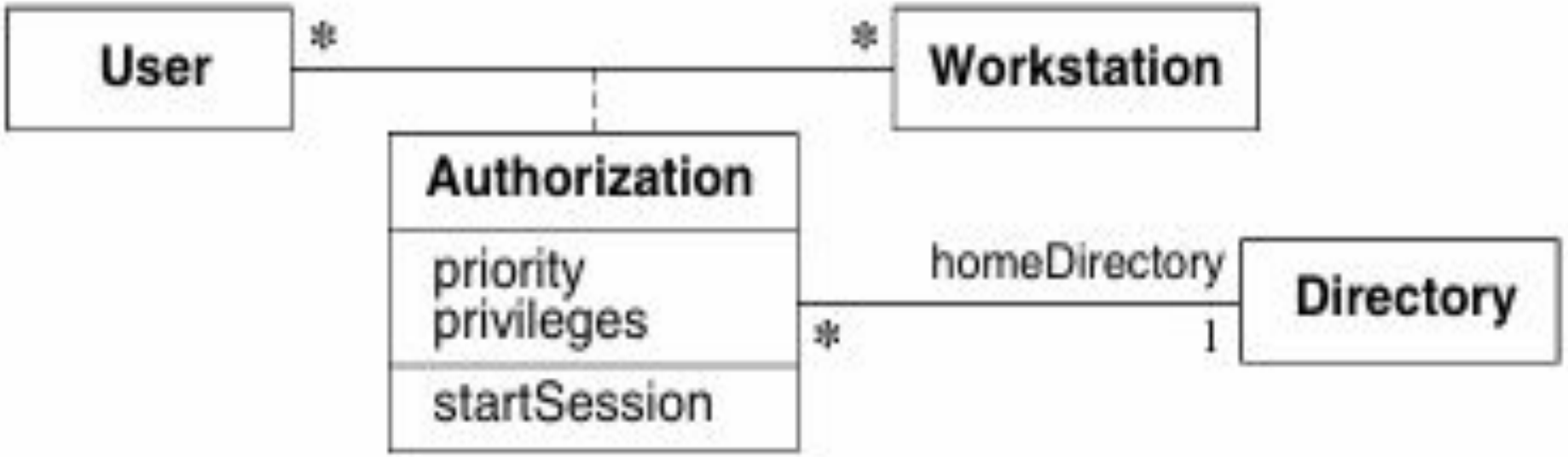


Fig: An association class participating in an association. Association classes let you specify identity and navigation paths precisely.

Qualified Associations

- In **Object-Oriented Modeling (OOM)**, a **qualified association** is a type of association where one of the associated classes has a **"key" or "qualifier"** that helps to uniquely identify a relationship or manage its multiplicity.

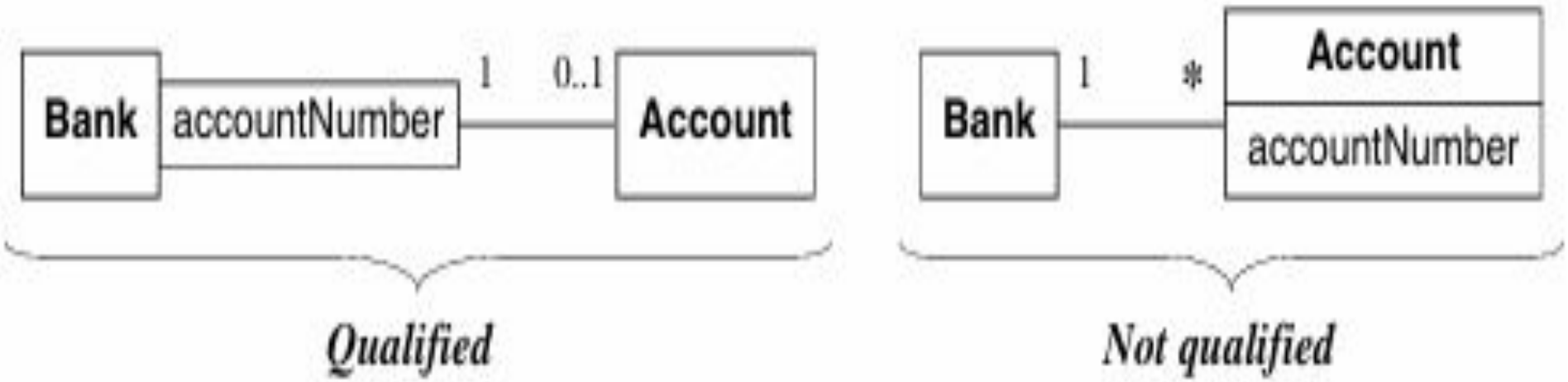
Key Characteristics of Qualified Associations:

- **Qualifier (Key)**: A **qualifier** is an attribute that serves as a key to uniquely identify the association. It is used to select or access specific instances of the associated class.

- **Simplifies Relationships**: It helps simplify relationships that would otherwise require additional associations or complex structures.

- **Reduces Redundancy**: In cases where you have a relationship with multiple instances, the qualifier eliminates the need for multiple associations by clearly defining how the related elements are linked.

- Figure below illustrates the most common use of a qualifier— for associations with one to-many multiplicity.
- A bank services multiple accounts. An account belongs to a single bank.
- Within the context of a bank, the account number specifies a unique account.
- Bank and Account are classes and accountNumber is the qualifier.
- Qualification reduces the effective multiplicity of this association from one-to-many to one-to-one.



- Both models are acceptable, but the qualified model adds information. The qualified model adds a multiplicity constraint, that the combination of a bank and an account number yields at most one account.

- The notation for a qualifier is a small box on the end of the association line near the source class. The qualifier box may grow out of any side (top, bottom, left, right) of the source class.

- The source class plus the qualifier yields the target class. In previous Figure Bank + accountNumber yields an Account, therefore accountNumber is listed in a box contiguous to Bank.

Generalization and Inheritance

- In **Object-Oriented Modeling (OOM)**, **generalization** and **inheritance** are two fundamental concepts that allow you to represent hierarchies and promote code reusability.

- These concepts are critical for organizing objects into meaningful relationships and managing their behavior and attributes efficiently.

- **Generalization** is the process of abstracting common characteristics (attributes and behaviors) from multiple classes into a more general, higher-level class. It is a way of forming hierarchies by identifying commonalities among classes and creating a superclass that contains these shared features.

- **Superclass**: The general class that contains common features shared by its subclasses.

- **Subclass**: A more specific class that inherits from the superclass and adds or modifies attributes and behaviors.

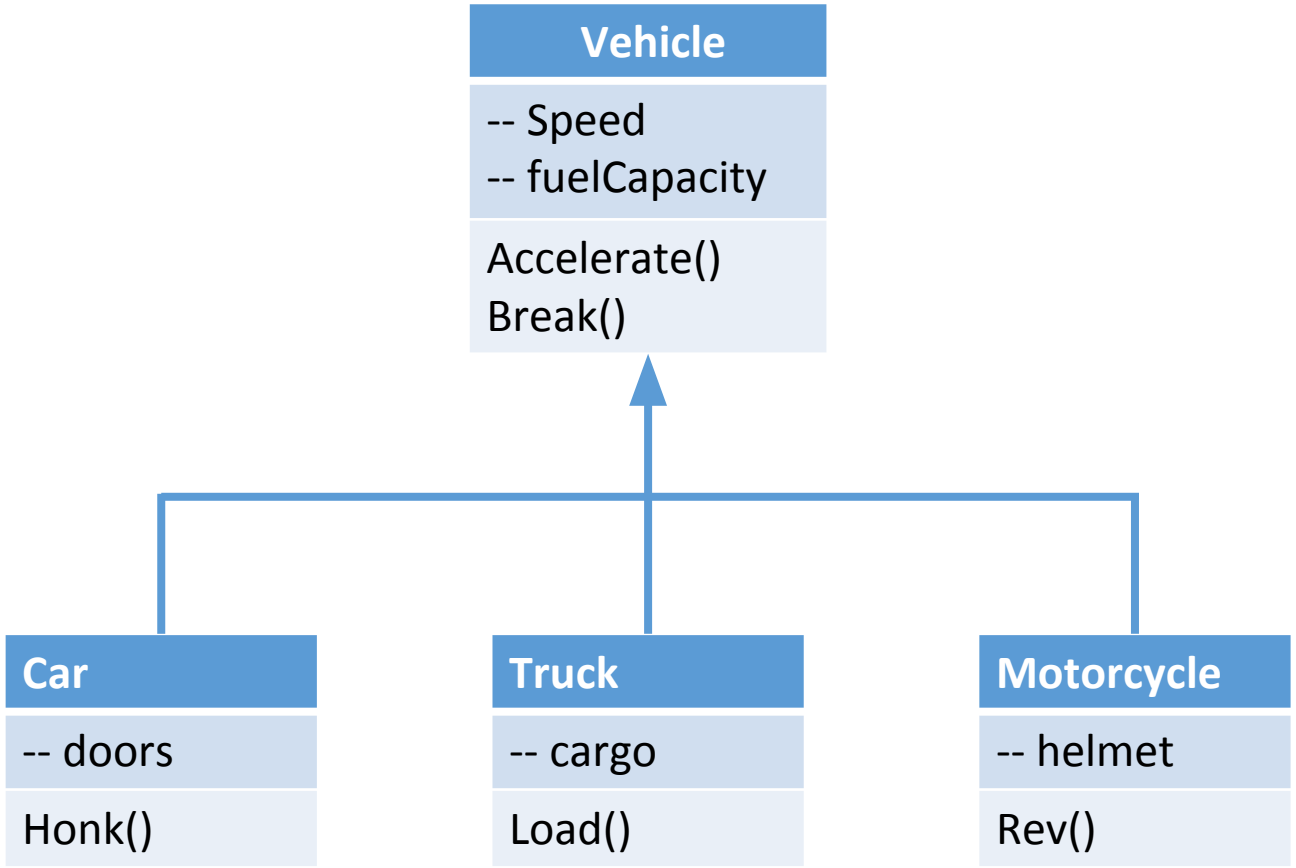
Example of Generalization:

- Let's say you're modeling a **vehicle system**:

 You have specific vehicle types like **Car**, **Truck**, and **Motorcycle**.

 All vehicles share common attributes like **speed** and **fuel capacity**, and behaviors like **accelerate()** and **brake()**.

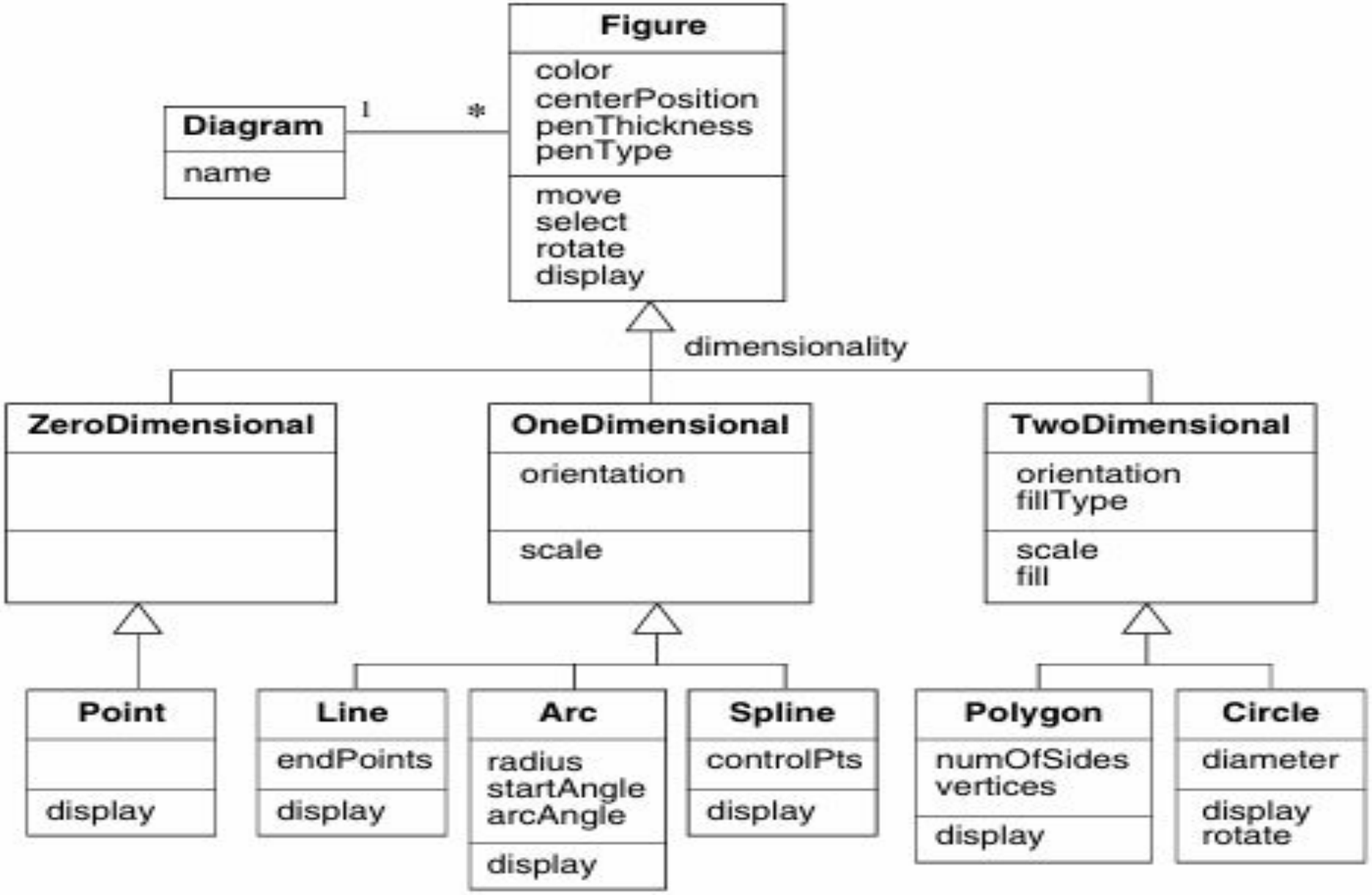
- You can create a general superclass called Vehicle, and the subclasses (Car, Truck, Motorcycle) will inherit common attributes and behaviors from Vehicle.



- Vehicle is the superclass (generalization), which contains attributes and methods common to all vehicles.

- Car, Truck, and Motorcycle are the subclasses (specializations), which inherit from Vehicle but can add specific features (like doors for a car, cargo for a truck, or helmet for a motorcycle).

- Figure below shows classes of geometric figures.
- This example has more of a programming flavor and emphasizes inheritance of operations.
- Move, select, rotate, and display are operations that all subclasses inherit.
- Scale applies to one-dimensional and two-dimensional figures.
- Fillapplies only to two-dimensional figures.
- The word written next to the generalization line in the diagram—dimensionality—is generalization set name.



Inheritance for graphic figures. Each subclass inherits the attributes, operations, and associations of its superclasses.

Use of Generalization

- **Facilitates Polymorphism**: Generalization provides the foundation for polymorphism in object-oriented systems. You can write code that works with general classes (superclasses) and process instances of their subclasses dynamically, allowing for more flexible and reusable code.

- **Promotes Code Reusability**: By extracting common attributes and behaviors into a superclass, you can avoid repeating the same code in multiple subclasses. This makes it easier to maintain and extend the system.

- **Provides a Natural Hierarchical Structure**: Generalization helps you create a hierarchy of classes, where higher-level classes represent general concepts and lower-level classes represent more specialized concepts. This hierarchical structure mirrors real-world relationships and enhances logical organization in the system.